PA2: UNIX Shell Pipes

PA1 and PA2

- PA2 is the next step of PA1
- The documents/codes of PA1 will be used for PA2
- We are building our own (**not fully functional**) shell (PA1)
- Parse command line, interpret, and execute command(s) (PA1)
- We will implement some features (>, < , >>,||, &&,|, ;)
  - logical operators, process control (PA1)
  - **Interprocess communication (IPC) via pipes (PA2)**
- Submission
  - Submit to Canvas a zip file
  - Folder should contain all your code, and a Makefile with an “all” target.

Evaluation

- Like all other programming assignments, graded on SoC servers
- Tests broken out with associated points
- Your shell should run in interactive mode
- It also needs to run with our testscripts (found in the handout directory)
- For each testscript we will run:

```
./osh -t < testscripts/testscript.txt > & tmp ; diff tmp
testscripts/ea.txt ;
```

Note that the “> &” redirects stderr to stdout and is unique to C-shell. For bash you would use “2>&1”

- Some output differences due to Unix version, type of parent shell
- Will dock points for:
  - infinite loops, not capable of running interactive shell mode, or you print extraneous output.
  - Check the provided binary in “cse-binary” to observe correct behavior.

Run csh shell

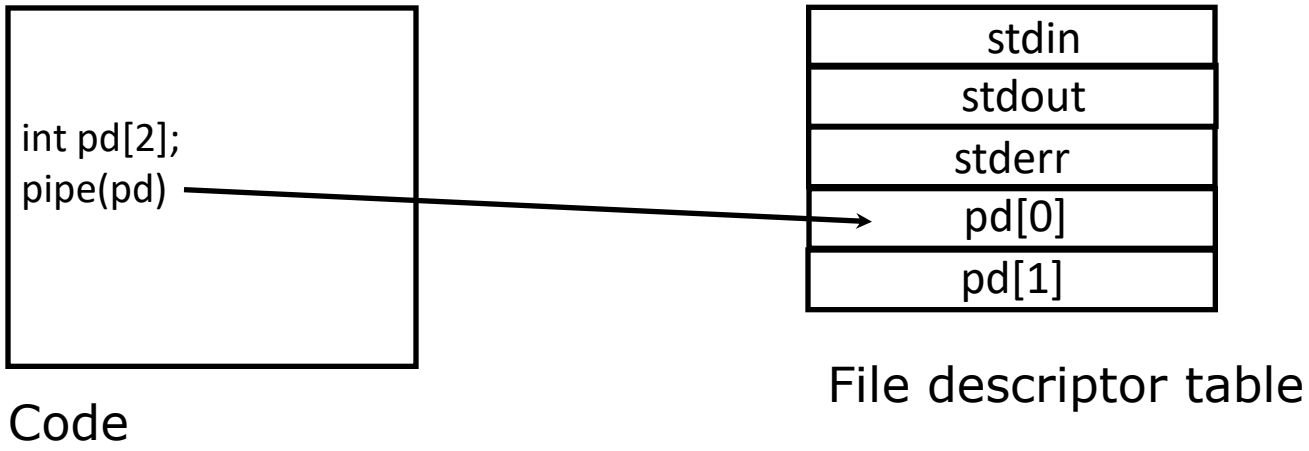
- To test your code, you can try to change bash shell to C-shell
- "exec csh" (or "exec tcsh", tcsh is an enhanced csh) and get a csh shell

Pipes

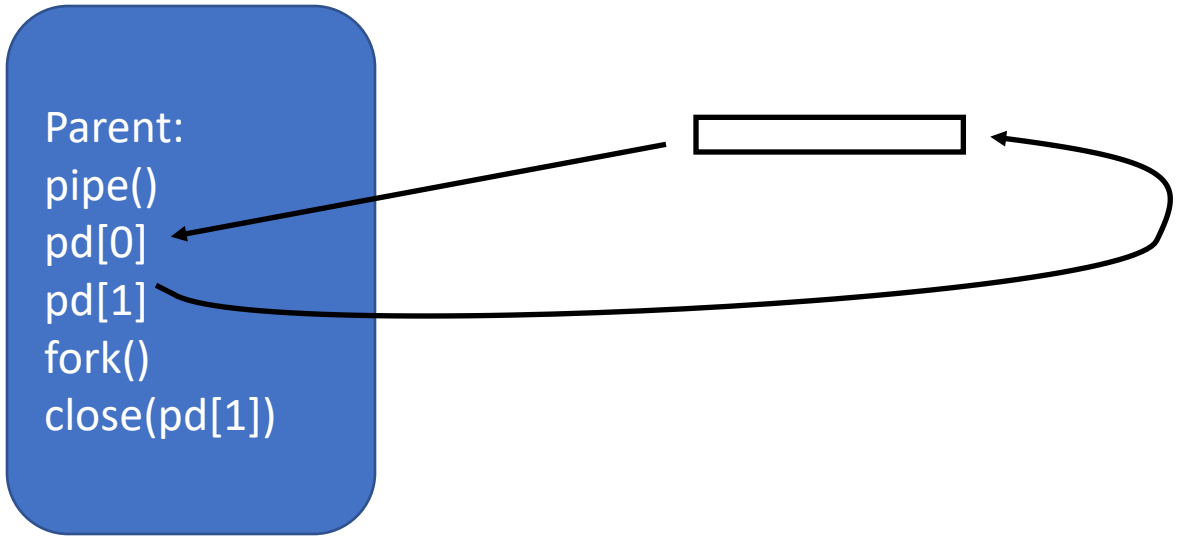
- pipes are created using the system call pipe()
- Like named pipes, pipes are a FIFO byte stream between two processes
- Unlike named pipe, pipes require a common process ancestor
- **Any process that writes to a pipe will hang until the data is read by another process.**
- **Any process that reads from a pipe will hang if there is no data in the pipe AND if there is any process with an open file handle to the pipe**

File descriptor table after pipe()

- Pipes are the oldest form of UNIX interprocess communication.
- They have the following limitations.
  - They are unidirectional
  - They are meant to be used between processes that have a common ancestor. ex: between parent and child, or between two children etc.
  - Why? Because file descriptors are copies from parent to child

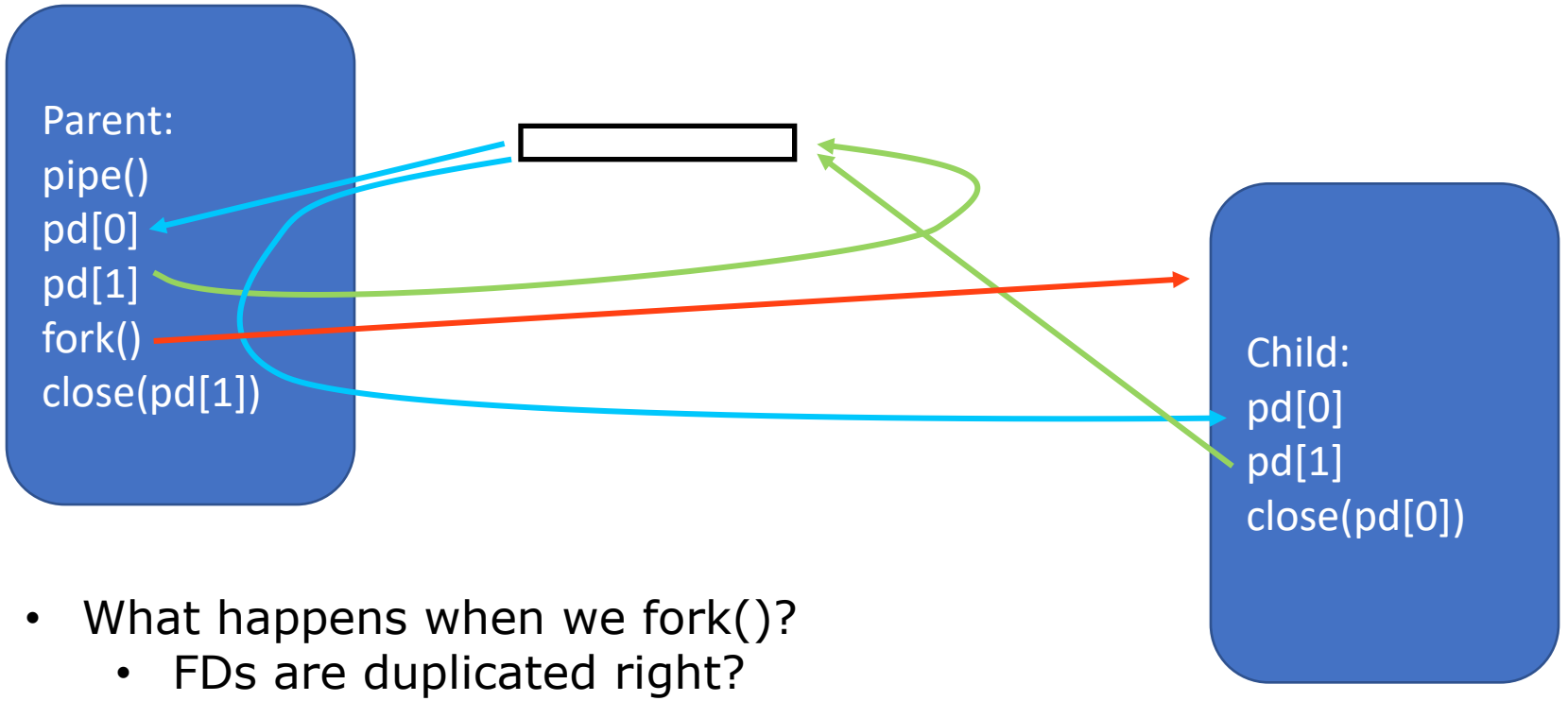


pipe in one process



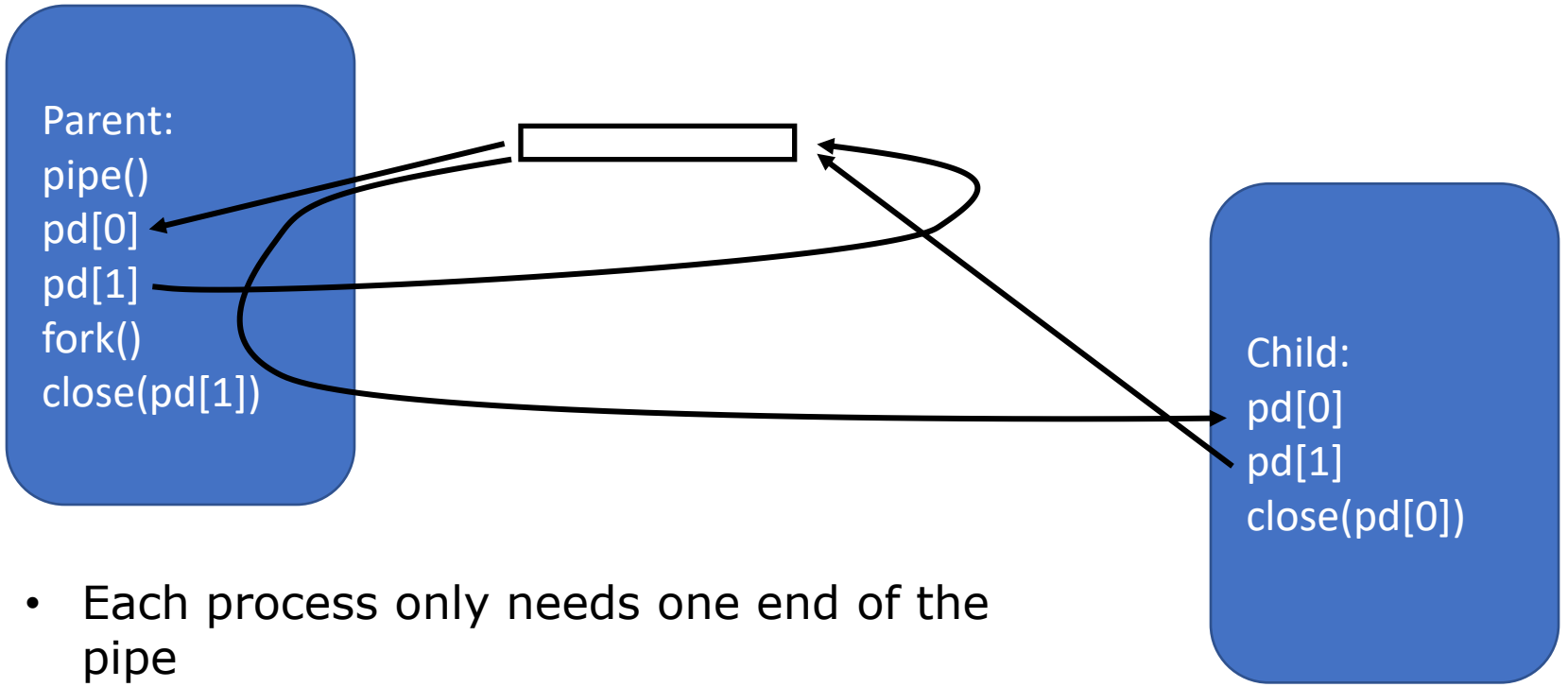
- File descriptors point to each end of the pipe

pipe between two processes - after fork



- What happens when we fork()?
  - FDs are duplicated right?
- Now both parent and child point to the same things.

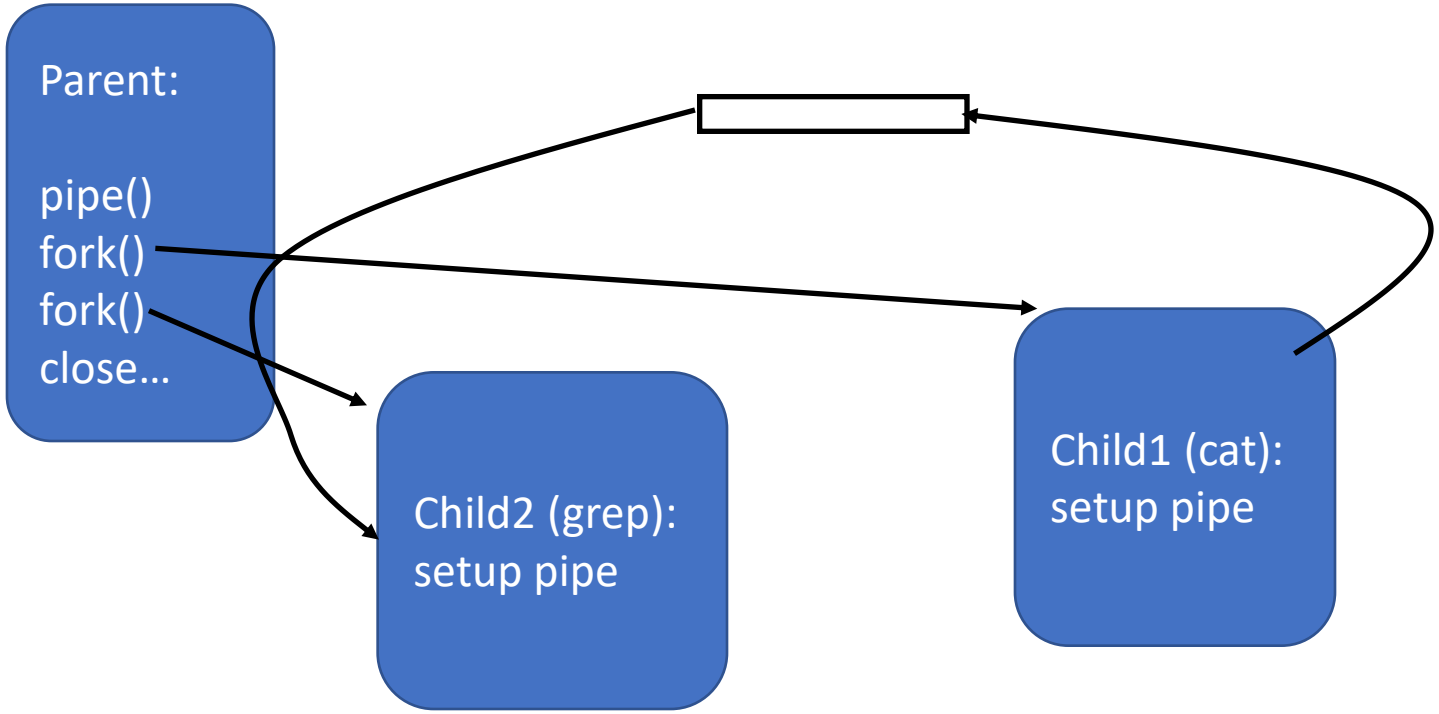
pipe between two processes - setup!



- Each process only needs one end of the pipe
- So close the correct end.

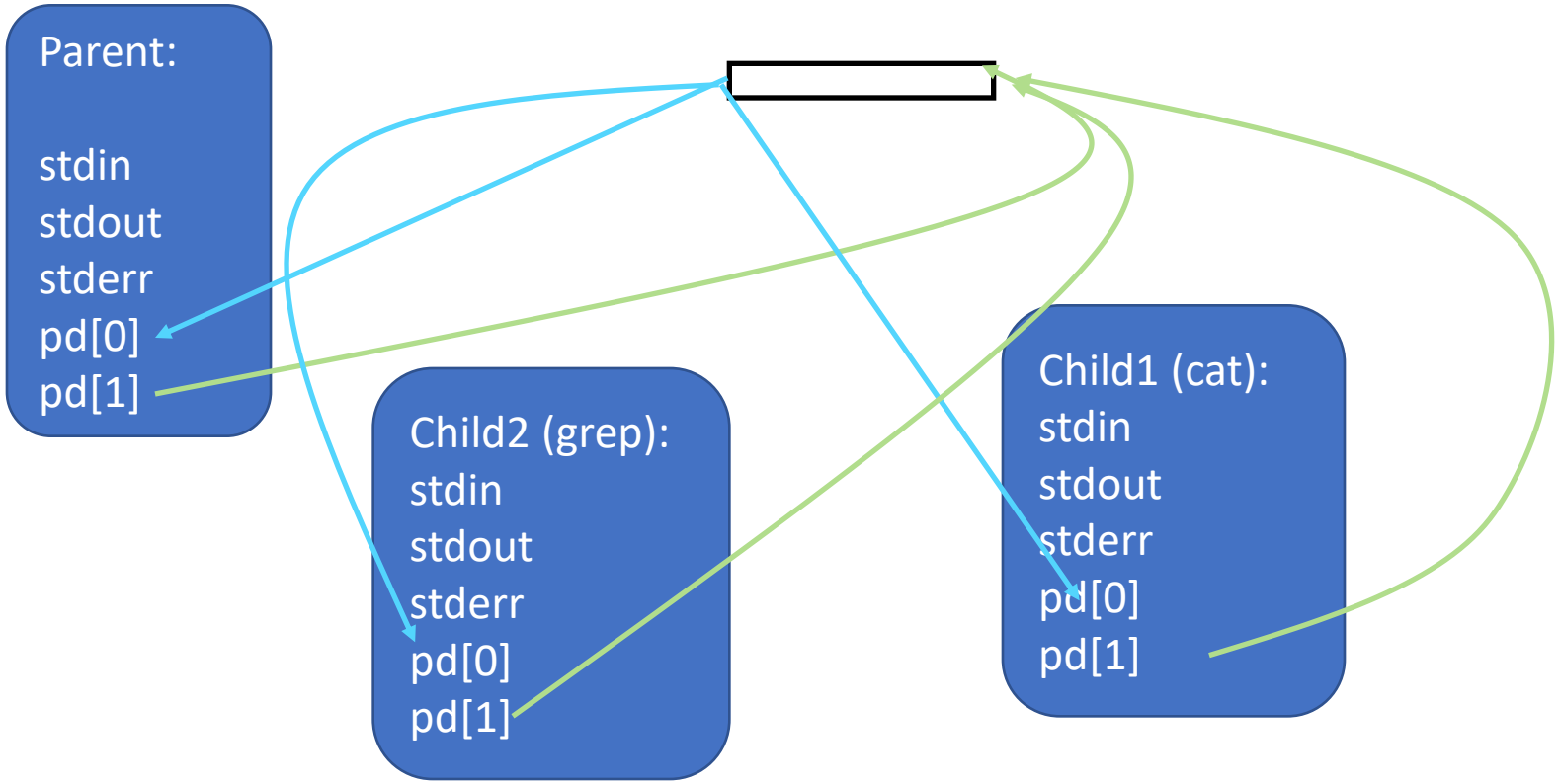
pipe between two commands

- cat /etc/motd | grep upgrade && echo “New software installed”
- Parent can only get exit status of child not grand children.
  - That means **parent has to fork both children**.



pipe between two commands (2)

- cat /etc/motd | grep upgrade && echo “New software installed”
- Parent can only get exit status of child not grand children.
  - That means **parent has to fork both children**.



pipe between two commands (1)

- cat /etc/motd | grep upgrade && echo “New software installed”
- Parent can only get exit status of child not grand children.
  - That means **parent has to fork both children**.

